

Java Stream API

(Complete Introduction & Key Concepts)



What is a Stream?



A sequence of elements from a source supporting pipeline processing.

- Focus on 'how' to process (declarative approach), not 'what' to store.

- Does not modify the source data. 
- Support for functional operations like map & filter.



The Absolute Basics: Collections vs. Streams

* Collections:

- Data-centric (store data)
- Eager evaluation (process all at once)
- Support repeated access.

* Streams:

- Process-centric (operate on data)
- Lazy evaluation (process only on demand)
- Consumed once (not reusable,  must create new stream).

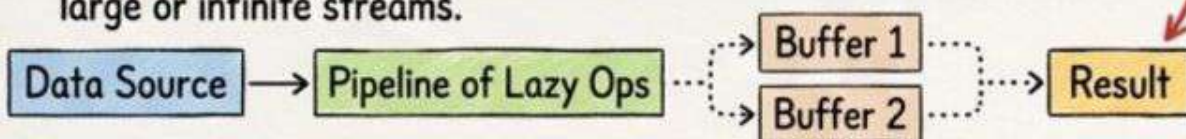


Under the Hood: Memory Architecture & Lazy Evaluation

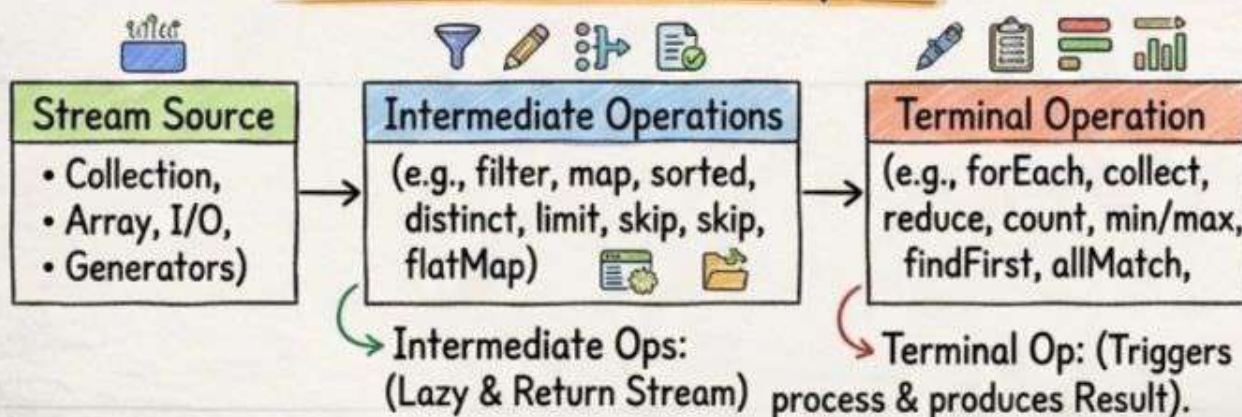
- Efficiency through Lazy Evaluation.
- **Lazy Evaluation:** Defining a pipeline of operations that are only executed when needed.
- **Memory Architecture:** Elements are processed 'just-in-time' and are not buffered or stored. Memory-efficient, suitable for very large or infinite streams.

Lazy Evaluation

Terminal Op triggers



Mechanics of a Stream Pipeline



Java Stream API

(Intermediate Operations & State Management)



The Heavyweights: map vs. flatMap

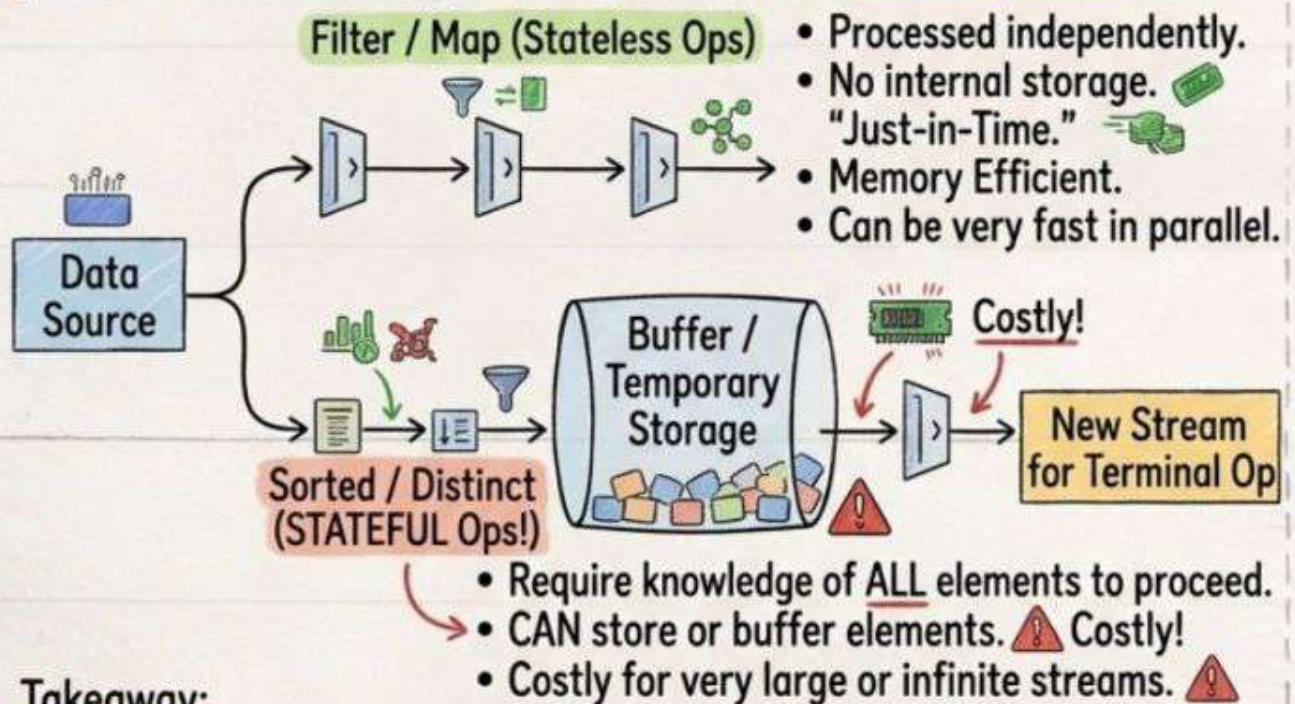


<u>map(): 1-to-1 Transformation</u> 	<u>flatMap(): 1-to-Many & Flattening</u>
Strings: ['Chandan', 'Amit'] $\xrightarrow{\text{map(String::toUpperCase)}$ Output: ['CHANDAN', 'AMIT'] Simple conversion.	Input: [['HELLO', 'WORLD', 'API']] $\xrightarrow{\text{flatMap(word} \rightarrow \text{letters_stream})}$ ['H'E', 'L'L', 'O'W', 'R'L', 'L'D', 'A', 'P'I'] stage: [['H'E'L'LO'], ['W'O'R'L'D'], ['A'P'I']] $\xrightarrow{\text{FLATTEN}}$ ['H'E', 'L'L', 'O'W', 'R'L', 'L'D', 'A', 'P'I'] Nested list to flat list.

- Applies a function to each element.
- Each input $\rightarrow$ 1 output.
- Elements stay on a single stream.
- Transforms elements into a stream and merges them.
- 1 input $\rightarrow$ many outputs.
- Useful for nested collections.



Under the Hood: Stateless vs. Stateful Operations



Takeaway:

State Management:
Always choose Stateless when possible for performance.

Stateful Bottleneck:
Sorted/Distinct can become a performance and memory issue.

Parallel Streams:
Stateful ops make parallelization harder

Java Stream API (Part 3)

(Terminal Operations & Data Aggregation)



reduce(): The Ultimate Aggregator



Collapse a stream into a single result, using an identity and an accumulator function.

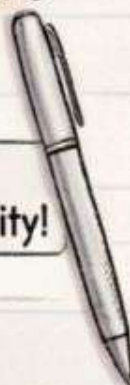
Input: `1, 2, 3` → `[(a,b) -> a+b]` →
$$\begin{array}{l} 0+1 = 1 \\ 1+2 = 3 \\ 3+3 = 6 \end{array}$$
 → Result: 6

String: `["A", "B"]` → `"AB"`

- Collapse stream into a single value.
- Uses an Identity & an Accumulator.
- Identity overloads: e.g., 0, "", false

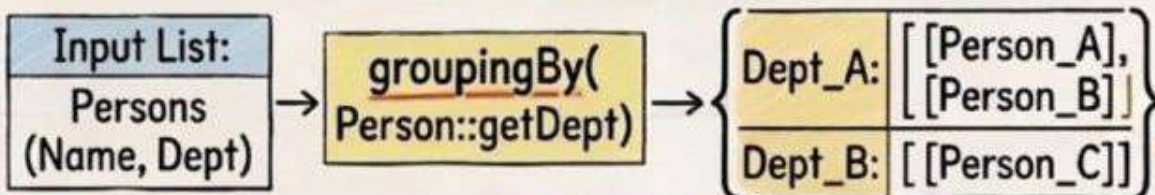


Safe for Infinite Streams with Identity!



Collectors.groupingBy(): The SQL of Java

Organizes elements into a Map based on a property.



- Creates a `Map<K, List<V>>`. SQL Equivalent: `'GROUP BY dept'`
- Downstream Collectors for advanced aggregation.
- Perfect for reports & summaries.



Other Common Terminal Ops

- forEach (Perform side-effects)
- count (Return number of elements)
- min/max (Find extreme values)
- findFirst/findAny (Short-circuiting, produce Optional)



Key Tip: Master Downstream Collectors to customize grouping summaries.

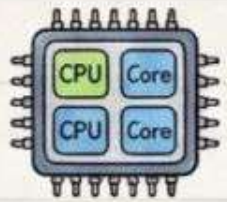
Java Stream API (Part 4)

Parallel Streams and Concurrency

1. What is a Parallel Stream? (Absolute Basics)

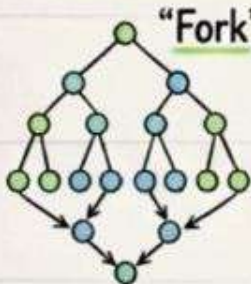
Splits stream into multiple sub-tasks processed concurrently on multiple CPU cores.

How? → Use `.parallelStream()` or `.stream().parallel()`



2. Under the Hood: The Fork/Join Framework

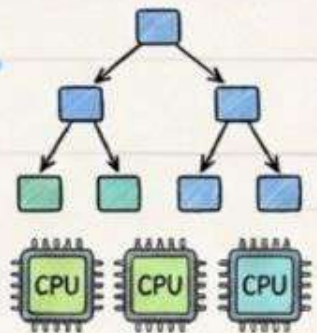
Uses a recursive decomposition strategy.



"Fork" Phase: Recursively splits task into smaller, independent sub-tasks until small enough.

"Compute" Phase: Process sub-tasks independently on different cores.

"Join" Phase: Combine sub-task results to form the final outcome.



3. The "Always Faster" Myth - Pitfalls & Reality



Sequential (Linear)

- predictable, single-thread flow
- lower overhead
- best for small datasets or non-compute tasks



Parallel (Multi-Core)

- potential speed-up
- higher overhead (context switching, splitting, merging)
- "high memory usage/cost"
- SLOWER for small tasks or high overhead!



4. Shared Mutable State (Race Conditions) - **DANGER!**

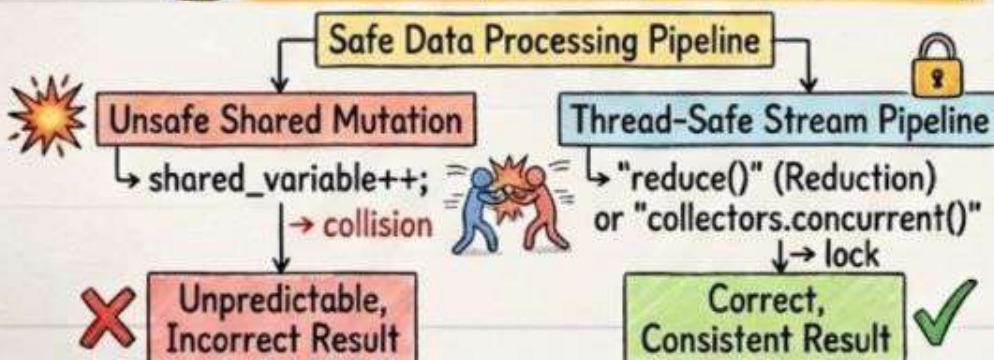


Multiple threads simultaneously modify shared non-thread-safe external state.

- Unpredictable and incorrect results.
- `"shared_variable.increment();" → Collision sketch`



Concurrency Management Hierarchy



Performance advice:
Only use Parallel Stream on large datasets with compute-intensive tasks. Avoid shared mutation. Measure results.